pro01-Nginx-Python-Flask-FastAPI-MySQL

Documentacion detallada del flujo completo

Comunicacion entre componentes, accion por accion

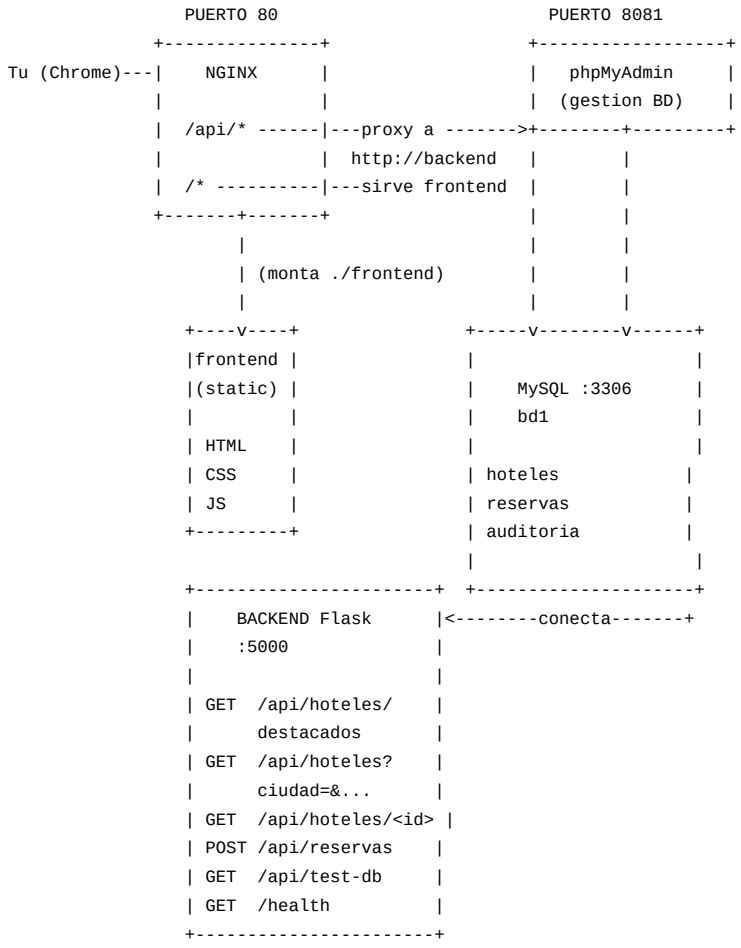
Arquitectura: Nginx + Flask (Python) + MySQL + phpMyAdmin

Frontend: HTML5 + CSS3 + JavaScript vanilla | Despliegue: Docker Compose

Indice

1. Esquema general de arquitectura
2. Accion 1: Cargar index.html y hoteles destacados
3. Accion 2: Rellenar formulario y pulsar Buscar
4. Accion 3: Cargar resultados.html (busqueda de hoteles)
5. Accion 4: Hacer clic en una tarjeta de hotel
6. Accion 5: Cargar detalle.html
7. Accion 6: Pulsar Confirmar reserva (validacion, SP, respuesta)
8. Mapa de navegacion completo

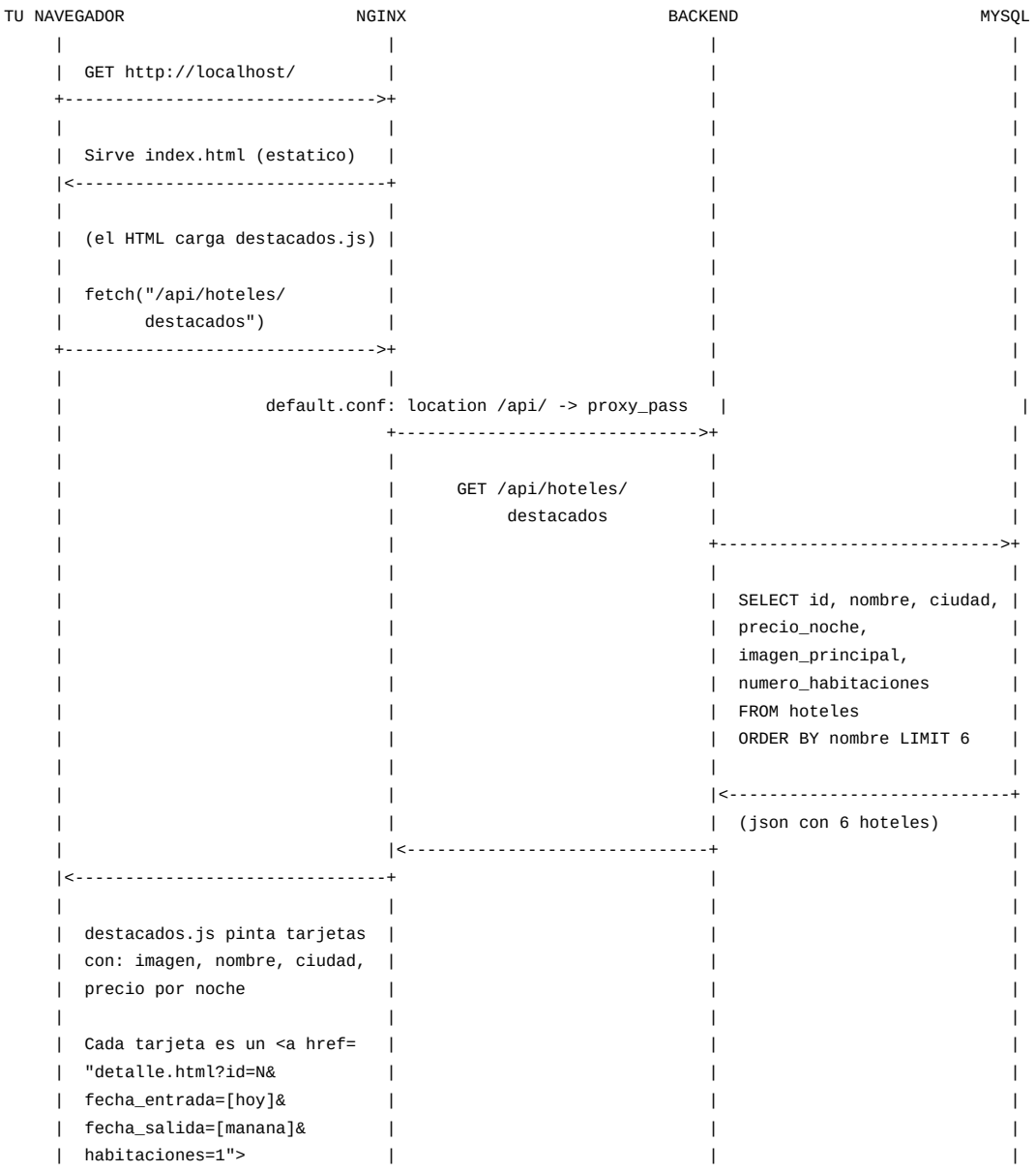
1. Esquema general de arquitectura



2. Accion 1: Cargar index.html (hoteles destacados)

Cuando entras en http://localhost, el navegador pide el index.html a Nginx. Nginx lo sirve directamente desde la carpeta ./frontend. El HTML incluye los archivos destacados.js y buscador.js. Al cargarse, destacados.js se ejecuta automaticamente y lanza una peticion a la API para obtener los hoteles destacados.

Diagrama de la comunicacion:



Resumen:

Sin que tu hagas nada mas que entrar en la pagina, el frontend ya se ha comunicado con el backend y ha pintado los 6 hoteles destacados. Cada tarjeta es un enlace que te lleva al detalle con fechas por defecto (hoy y manana, 1 habitacion).

3. Accion 2: Rellenar formulario y pulsar Buscar

El formulario de busqueda tiene 4 campos: ciudad (texto), fecha de entrada, fecha de salida y numero de habitaciones. El archivo buscador.js gestiona el evento submit del formulario.

Diagrama (solo frontend, no hay llamada a API):

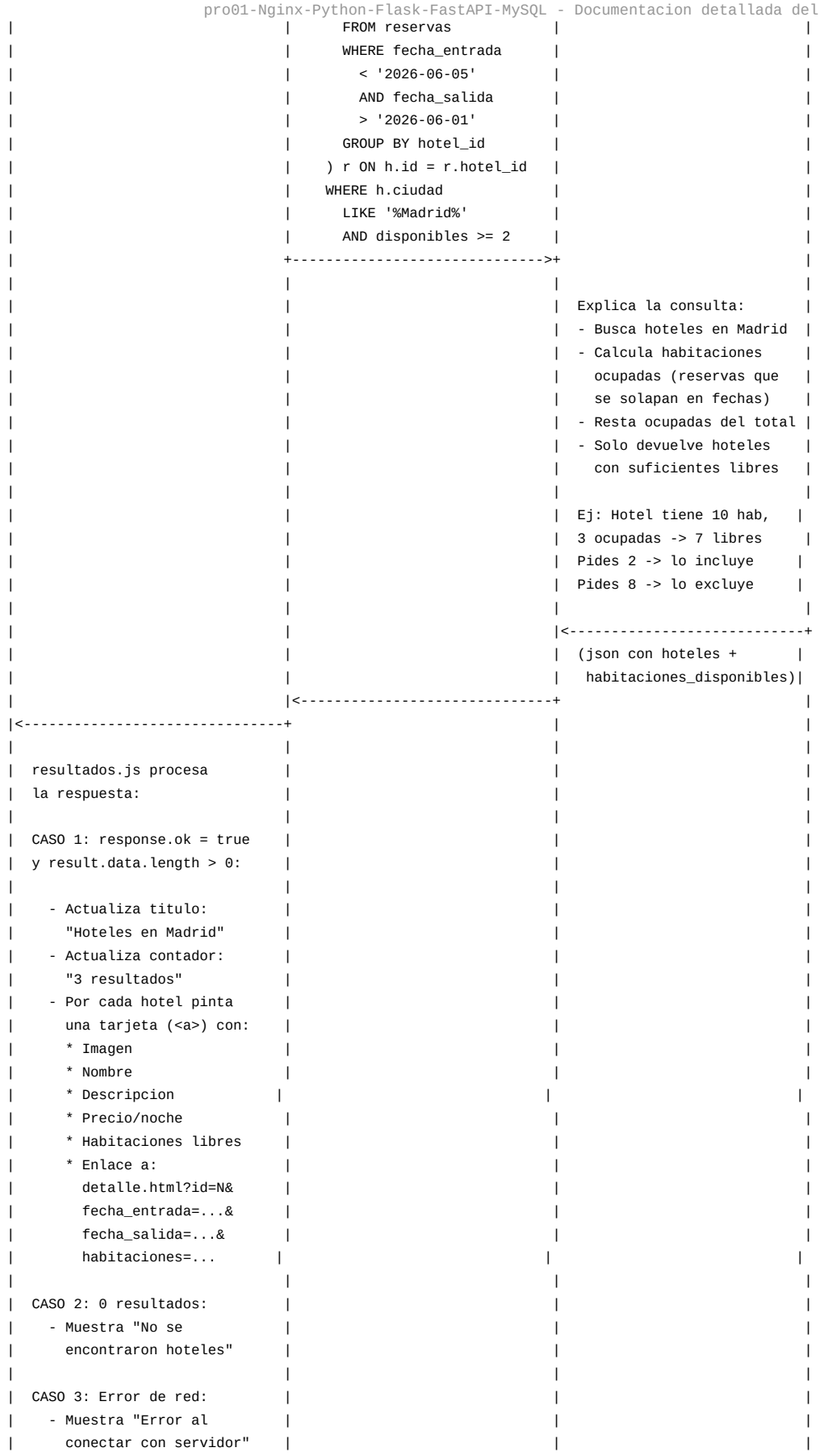
```
TU NAVEGADOR
     |
     |  (buscador.js - validacion en cliente)
     |  1. e.preventDefault() -> evita recarga normal
     |
     |  2. Lee valores de los campos:
     |     - ciudad = "Madrid"
     |     - fecha_entrada = "2026-06-01"
     |     - fecha_salida = "2026-06-05"
     |     - habitaciones = "2"
     |
     |  3. Validaciones:
     |     - Todos los campos tienen valor?
     |       NO -> alert("Todos los campos son obligatorios.")
     |       SI -> continuar
     |
     |     - Fecha salida > fecha entrada?
     |       NO -> alert("La fecha de salida debe ser posterior...")
     |       SI -> continuar
     |
     |  4. Construye URL con parametros:
     |     resultados.html?ciudad=Madrid
     |                     &fecha_entrada=2026-06-01
     |                     &fecha_salida=2026-06-05
     |                     &habitaciones=2
     |
     |  5. Redirige navegador:
     |     window.location.href = "..."
     |
     +------------------------------------------------>
```

Resumen:

Todo ocurre en el navegador del cliente. No se llama al backend ni a la BD. Solo se validan los campos y se redirige a resultados.html con los filtros en la URL. El backend se usara en el siguiente paso, al cargar resultados.html.

4. Accion 3: Cargar resultados.html (busqueda de hoteles)

Al llegar a resultados.html, el archivo resultados.js se ejecuta. Lee los parametros de la URL y hace una peticion a la API con los filtros.


FROM reservas
WHERE fecha_entrada
< '2026-06-05'
AND fecha_salida
> '2026-06-01'
GROUP BY hotel_id
) r ON h.id = r.hotel_id
WHERE h.ciudad
LIKE '%Madrid%'
AND disponibles >= 2
Explica la consulta:
- Busca hoteles en Madrid
- Calcula habitaciones ocupadas (reservas que se solapan en fechas)
- Resta ocupadas del total
- Solo devuelve hoteles con suficientes libres
Ej: Hotel tiene 10 hab, 3 ocupadas -> 7 libres
Pides 2 -> lo incluye
Pides 8 -> lo excluye
(json con hoteles + habitaciones_disponibles)
resultados.js procesa la respuesta:
CASO 1: response.ok = true y result.data.length > 0:
- Actualiza titulo: "Hoteles en Madrid"
- Actualiza contador: "3 resultados"
- Por cada hotel pinta una tarjeta (<a>) con:
* Imagen
* Nombre
* Descripcion
* Precio/noche
* Habitaciones libres
* Enlace a: detalle.html?id=N&fecha_entrada=...&fecha_salida=...&habitaciones=...
CASO 2: 0 resultados:
- Muestra "No se encontraron hoteles"
CASO 3: Error de red:
- Muestra "Error al conectar con servidor"

5. Accion 4: Hacer clic en una tarjeta de hotel

Tanto en los destacados (index.html) como en los resultados (resultados.html), cada tarjeta de hotel es un enlace HTML normal.

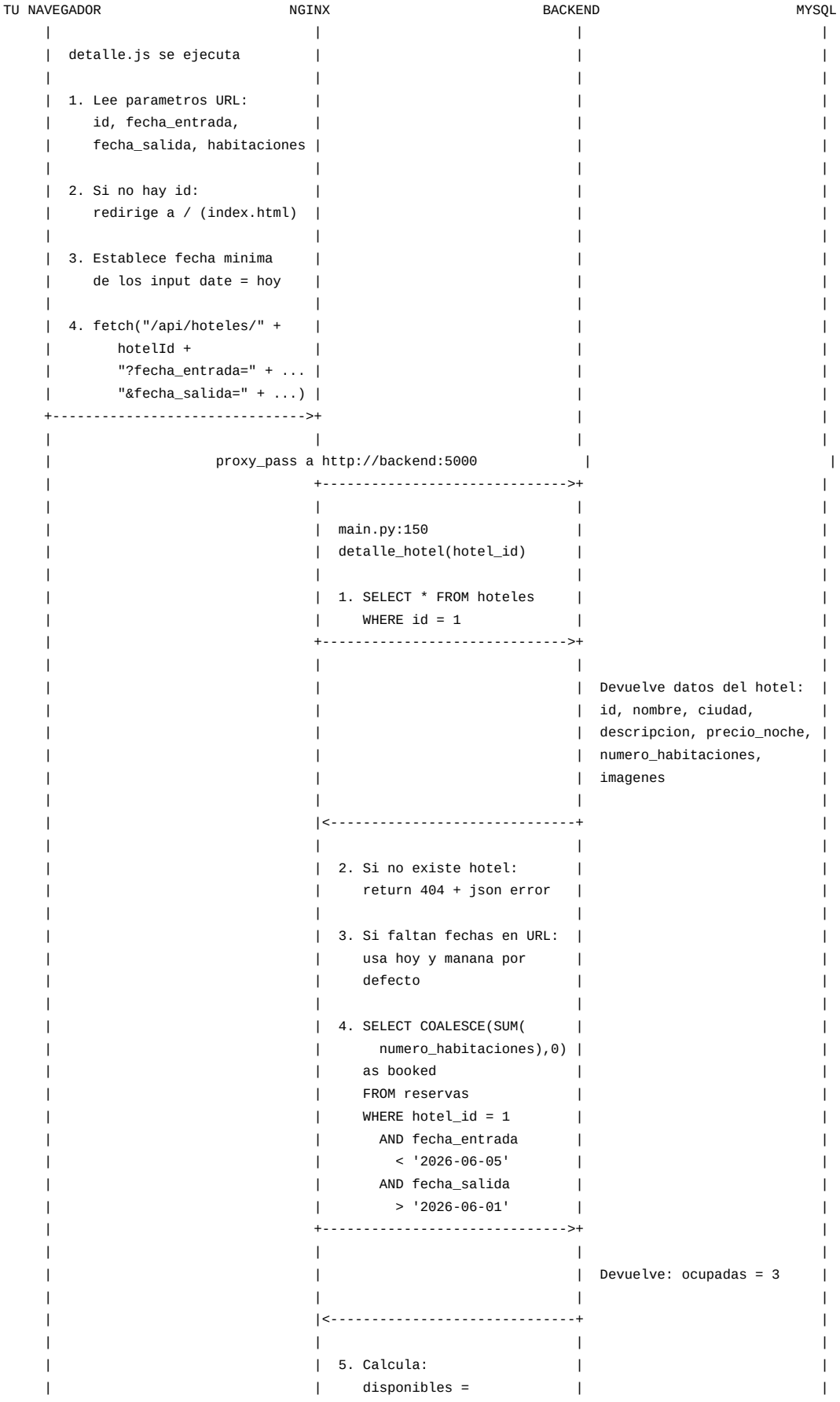
```
TU NAVEGADOR
     |
     |  Las tarjetas se renderizan como:
     |
     |  <a href="detalle.html?id=1
     |         &fecha_entrada=2026-06-01
     |         &fecha_salida=2026-06-05
     |         &habitaciones=2">
     |    [imagen del hotel]
     |    [nombre del hotel]
     |    [precio]
     |  </a>
     |
     |  Cuando haces CLIC en la tarjeta:
     |
     |  El navegador navega a:
     |  detalle.html?id=1
     |             &fecha_entrada=2026-06-01
     |             &fecha_salida=2026-06-05
     |             &habitaciones=2
     |
     +------------------------------------------------>
     |
     |  NO hay peticion AJAX. Es una navegacion
     |  normal del navegador (window.location).
     |  Los parametros viajan en la URL para que
     |  detalle.html los lea al cargarse.
```

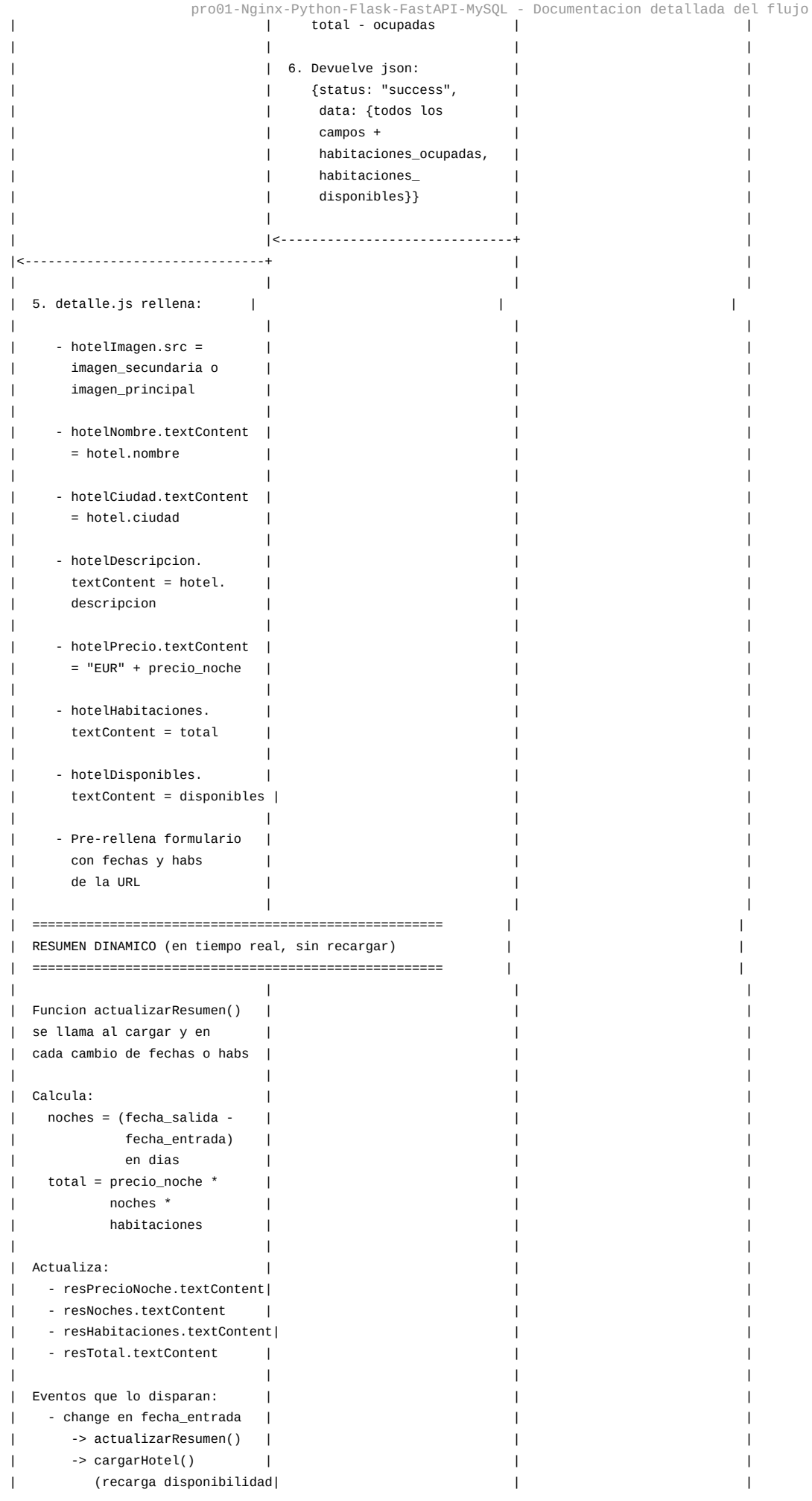
Resumen:

Es un enlace HTML normal. No hay llamada a API en este paso. Los datos del hotel y las fechas se pasan mediante parametros en la URL para que la pagina detalle.html los recoja al cargar.

6. Accion 5: Cargar detalle.html

La pagina detalle.html es la mas compleja. Al cargarse, detalle.js se ejecuta, lee el id del hotel de la URL y pide todos los datos al backend.




total - ocupadas
6. Devuelve json:
{status: "success",
data: {todos los
campos +
habitaciones_ocupadas,
habitaciones_
disponibles}}
5. detalle.js rellena:
- hotelImagen.src =
imagen_secundaria o
imagen_principal
- hotelNombre.textContent
= hotel.nombre
- hotelCiudad.textContent
= hotel.ciudad
- hotelDescripcion.
textContent = hotel.
descripcion
- hotelPrecio.textContent
= "EUR" + precio_noche
- hotelHabitaciones.
textContent = total
- hotelDisponibles.
textContent = disponibles
- Pre-rellena formulario
con fechas y habs
de la URL
RESUMEN DINAMICO (en tiempo real, sin recargar)
Funcion actualizarResumen()
se llama al cargar y en
cada cambio de fechas o habs
Calcula:
noches = (fecha_salida -
fecha_entrada)
en dias
total = precio_noche *
noches *
habitaciones
Actualiza:
- resPrecioNoche.textContent
- resNoches.textContent
- resHabitaciones.textContent
- resTotal.textContent
Eventos que lo disparan:
- change en fecha_entrada
-> actualizarResumen()
-> cargarHotel()
(recarga disponibilidad

```
|           si cambias fechas)   |                              |                              |
|    - change en fecha_salida    |                              |                              |
|       -> actualizarResumen()   |                              |                              |
|       -> cargarHotel()         |                              |                              |
|    - input en habitaciones     |                              |                              |
|       -> actualizarResumen()   |                              |                              |
```

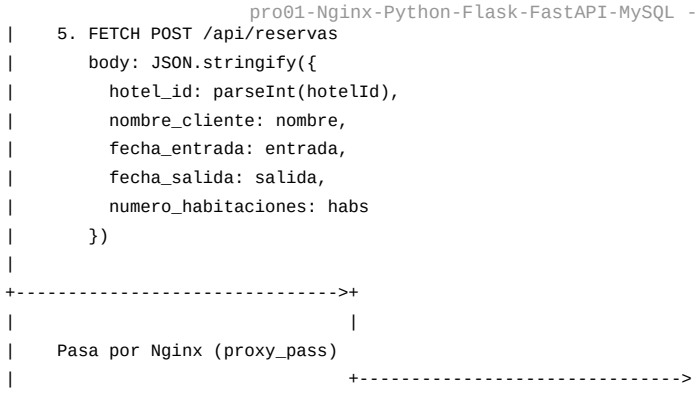
7. Accion 6: Pulsar Confirmar reserva

Esta es la accion mas importante y compleja. Involucra validacion en cliente, envio HTTP, validacion en servidor, logica de negocio en un Stored Procedure de MySQL, un trigger de auditoria, y respuesta al usuario.

FASE 1 - Validacion en el cliente (detalle.js)

```
TU NAVEGADOR
     |
     |  El formulario tiene 4 campos:
     |  [nombre_cliente] (texto)
     |  [fecha_entrada]  (date)
     |  [fecha_salida]   (date)
     |  [numero_habitaciones] (number)
     |
     |  Al pulsar "Confirmar reserva":
     |
     |  form.addEventListener("submit", async function(e) {
     |    e.preventDefault(); // no recargar pagina
     |
     |    1. Ocultar errores anteriores
     |    2. Validar cada campo:
     |
     |       nombre = inputNombre.value.trim()
     |       SI nombre vacio ->
     |         marcarError(inputNombre, errorNombre)
     |         valido = false
     |
     |       entrada = inputEntrada.value
     |       SI entrada vacia ->
     |         marcarError(inputEntrada, errorEntrada)
     |         valido = false
     |
     |       salida = inputSalida.value
     |       SI salida vacia ->
     |         marcarError(inputSalida, errorSalida)
     |         valido = false
     |
     |       habs = parseInt(inputHabitaciones.value)
     |       SI !habs o habs < 1 ->
     |         marcarError(inputHabitaciones,
     |           errorHabitaciones)
     |         valido = false
     |
     |    3. Validaciones extra:
     |
     |       SI salida <= entrada ->
     |         alertContainer.innerHTML = error
     |         return
     |
     |       SI entrada < today ->
     |         alertContainer.innerHTML = error
     |         return
     |
     |       SI habs > disponibles ->
     |         alertContainer.innerHTML =
     |           "Solo hay X disponibles"
     |         return
     |
     |    4. Si todo valido:
     |       - btnReservar.disabled = true
     |       - btnReservar.textContent = "Reservando..."
```

FASE 2 - Envio al backend


5. FETCH POST /api/reservas
body: JSON.stringify({
hotel_id: parseInt(hotelId),
nombre_cliente: nombre,
fecha_entrada: entrada,
fecha_salida: salida,
numero_habitaciones: habs
})
Pasa por Nginx (proxy_pass)

FASE 3 - Validacion en el backend (Flask)

```
BACKEND (main.py:192 - crear_reserva())
     |
     |  Recibe JSON del body
     |
     |  1. Validar campos obligatorios:
     |     for field in required:
     |       if field not in data -> 400
     |
     |  2. Validar formatos:
     |     hotel_id = int(data["hotel_id"])
     |     habitaciones = int(data["numero_habitaciones"])
     |     fecha_entrada = datetime.strptime(...)
     |     fecha_salida = datetime.strptime(...)
     |     SI error de formato -> 400
     |
     |  3. Validar reglas de negocio:
     |     SI salida <= entrada -> 400
     |     SI entrada < hoy -> 400
     |     SI habitaciones <= 0 -> 400
     |     SI nombre > 100 chars -> 400
     |
     |  4. Conectar a MySQL:
     |     connection = get_db()
     |     cursor = connection.cursor()
     |
     |  5. Ejecutar STORED PROCEDURE:
     |
     |  CALL sp_realizar_reserva(
     |     1,                    -- hotel_id
     |     'Marco',              -- nombre_cliente
     |     '2026-06-01',         -- fecha_entrada
     |     '2026-06-05',         -- fecha_salida
     |     2,                    -- numero_habitaciones
     |     @p_res,               -- OUT: resultado (1=ok, 0=error)
     |     @p_msg                -- OUT: mensaje
     |  )
     |
     +---------------------------------------------------------------->
```

FASE 4 - Stored Procedure en MySQL (logica de negocio)

```
MYSQL - Stored Procedure sp_realizar_reserva
(Definido en init.sql lineas 41-81)
     |
     |  CREATE PROCEDURE sp_realizar_reserva(
     |    IN p_hotel_id INT,
     |    IN p_nombre_cliente VARCHAR(100),
     |    IN p_fecha_entrada DATE,
     |    IN p_fecha_salida DATE,
     |    IN p_numero_habitaciones INT,
     |    OUT p_resultado INT,
     |    OUT p_mensaje VARCHAR(255)
     |  )
     |  BEGIN
     |
     |    -- 1. Obtener total de habitaciones del hotel
     |    SELECT numero_habitaciones
     |    INTO v_total_habitaciones
     |    FROM hoteles WHERE id = p_hotel_id;
     |
     |    -- Ejemplo: v_total_habitaciones = 10
     |
     |    -- 2. Si el hotel no existe -> ERROR
     |    IF v_total_habitaciones IS NULL THEN
     |      SET p_resultado = 0;
```

```
|      SET p_mensaje = 'Hotel no encontrado';
|
|    ELSE
|
|      -- 3. Calcular habitaciones ocupadas
|      --    en el rango de fechas solicitado
|      SELECT COALESCE(SUM(numero_habitaciones),0)
|      INTO v_ocupadas
|      FROM reservas
|      WHERE hotel_id = p_hotel_id
|        AND fecha_entrada < p_fecha_salida
|        AND fecha_salida > p_fecha_entrada;
|
|      -- Ejemplo: v_ocupadas = 3
|
|      -- 4. Calcular disponibles
|      SET v_disponibles = v_total_habitaciones
|                        - v_ocupadas;
|      -- Ejemplo: v_disponibles = 10 - 3 = 7
|
|      -- 5. Hay suficientes habitaciones?
|      IF v_disponibles < p_numero_habitaciones THEN
|        -- NO: error
|        SET p_resultado = 0;
|        SET p_mensaje = CONCAT(
|          'No hay suficientes. Disponibles: ',
|          v_disponibles);
|      ELSE
|        -- SI: crear la reserva
|        INSERT INTO reservas (
|          hotel_id, nombre_cliente,
|          fecha_entrada, fecha_salida,
|          numero_habitaciones
|        ) VALUES (
|          p_hotel_id, p_nombre_cliente,
|          p_fecha_entrada, p_fecha_salida,
|          p_numero_habitaciones
|        );
|
|        -- EXITO
|        SET p_resultado = 1;
|        SET p_mensaje = CONCAT(
|          'Reserva creada correctamente. ID: ',
|          LAST_INSERT_ID());
|      END IF;
|    END IF;
|  END
|
|  ============================================
|  TRIGGER AUTOMATICO (se dispara solo)
|  ============================================
|
|  CREATE TRIGGER tr_auditoria_reservas
|  AFTER INSERT ON reservas
|  FOR EACH ROW
|  BEGIN
|    INSERT INTO auditoria_reservas (
|      hotel_id, reserva_id, fecha_hora
|    ) VALUES (
|      NEW.hotel_id, NEW.id, NOW()
|    );
|  END
|
|  -> Cuando se hace el INSERT en reservas,
|     este trigger se ejecuta AUTOMATICAMENTE
|     y registra en auditoria_reservas:
|     - hotel_id = 1
|     - reserva_id = 5 (el ID recien creado)
|     - fecha_hora = 2026-05-26 15:30:00
```

FASE 5 - Retorno al backend y respuesta final

```
BACKEND (main.py:192 - crear_reserva()) - continuacion
     |
     |  Despues de CALL sp_realizar_reserva(...):
     |
     |  cursor.execute("SELECT @p_res, @p_msg")
     |  row = cursor.fetchone()
     |
     |  p_resultado = row[0]  -- 1 o 0
     |  p_mensaje   = row[1]  -- texto
     |
     |  SI p_resultado == 1:
     |    -- Extraer ID de reserva del mensaje
     |    -- con regex: r'ID:\s*(\d+)'
     |    reserva_id = int(match.group(1))
     |
     |    connection.commit()
     |
     |    return jsonify({
     |      "status": "success",
     |      "message": "Reserva creada correctamente",
     |      "data": {"id": reserva_id}
     |    }), 201
     |
     |  SI NO:
     |    connection.rollback()
     |
     |    return jsonify({
     |      "status": "error",
     |      "message": p_mensaje
     |    }), 400
     |
     |  SI hay excepcion mysql.connector.Error:
     |    connection.rollback()
     |    logger.error("Error BD: %s", exc)
     |    return jsonify({...}), 500
     |
     |  FINALLY:
     |    cerrar conexion si esta abierta
     |
     |<------------------------------------------------------------+
```

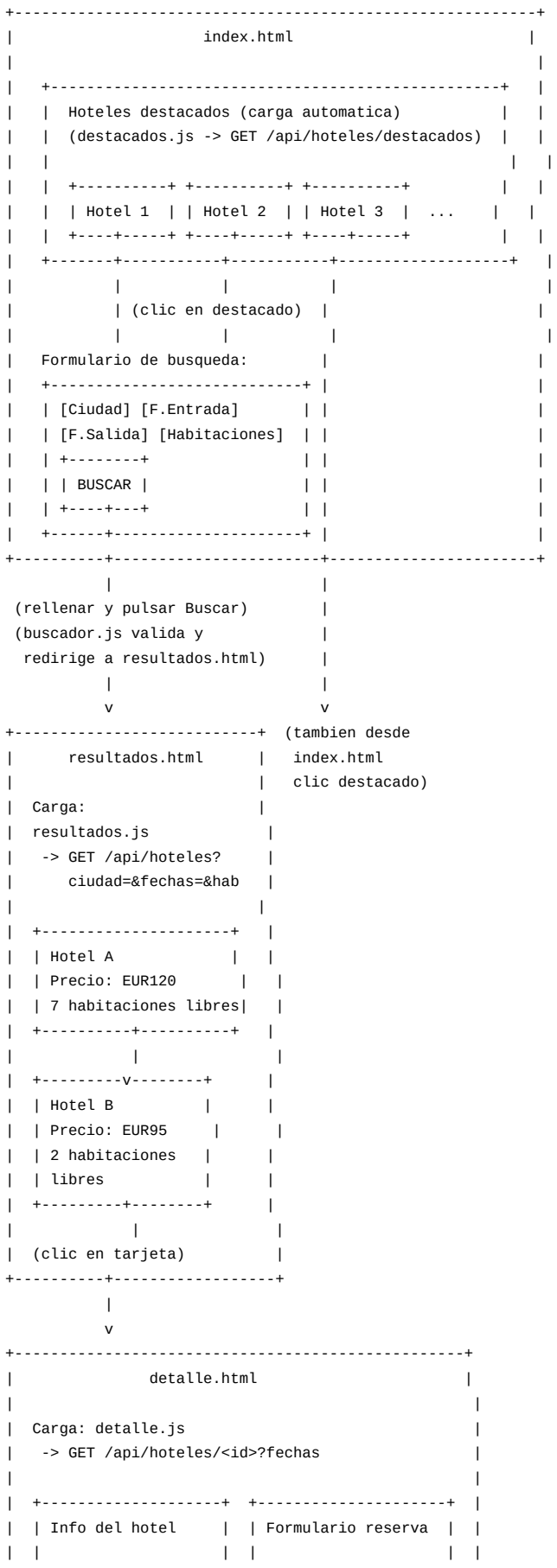
FASE 6 - Respuesta al usuario (frontend)

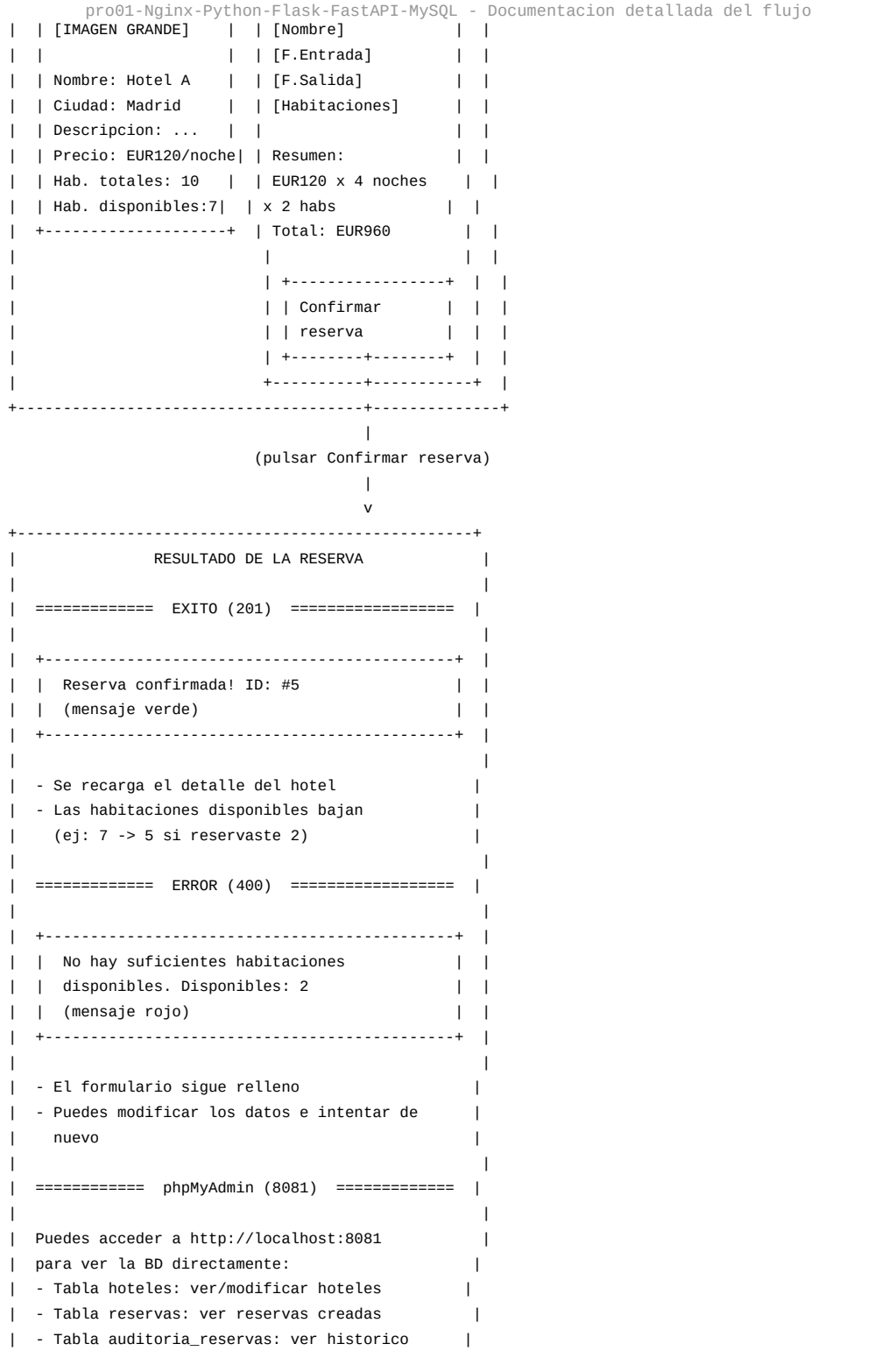
```
TU NAVEGADOR (detalle.js - continuacion)
     |
     |  try {
     |    response = await fetch("/api/reservas", {...})
     |    result = await response.json()
     |
     |    if (response.ok) { // codigo 201
     |
     |      -- Actualizar URL del navegador
     |         (sin recargar, con history.replaceState)
     |
     |      -- MOSTRAR MENSAJE VERDE:
     |         alertContainer.innerHTML =
     |           '<div class="alert alert-success">'
     |           + 'Reserva confirmada! ID: #5'
     |           + '</div>'
     |
     |      -- Mantener fechas en formulario
     |         (se hace form.reset() y se re-asignan)
     |
     |      -- RECARGAR detalle del hotel:
     |         await cargarHotel()
```

```
|         (las habitaciones disponibles
|          se actualizan: bajan en 2)
|
|    } else { // codigo 400
|
|      -- MOSTRAR MENSAJE ROJO:
|         alertContainer.innerHTML =
|           '<div class="alert alert-error">'
|           + "No hay suficientes. Disp: 2"
|           + '</div>'
|    }
|
|  } catch (error) { // error de red
|
|    -- MOSTRAR MENSAJE ROJO:
|       alertContainer.innerHTML =
|         '<div class="alert alert-error">'
|         + "Error al conectar con servidor"
|         + '</div>'
|  }
|
|  finally {
|    -- Reactivar boton
|    btnReservar.disabled = false
|    btnReservar.textContent = "Confirmar reserva"
|  }
```

8. Mapa de navegacion completo

A continuacion se muestra el mapa visual de todas las pantallas y acciones posibles dentro de la aplicacion web:




[IMAGEN GRANDE]
Nombre: Hotel A
Ciudad: Madrid
Descripcion: ...
Precio: EUR120/noche
Hab. totales: 10
Hab. disponibles:7
[Nombre]
[F.Entrada]
[F.Salida]
[Habitaciones]
Resumen:
EUR120 x 4 noches
x 2 habs
Total: EUR960
Confirmar reserva
(pulsar Confirmar reserva)
RESULTADO DE LA RESERVA
EXITO (201)
Reserva confirmada! ID: #5
(mensaje verde)
- Se recarga el detalle del hotel
- Las habitaciones disponibles bajan
(ej: 7 -> 5 si reservaste 2)
ERROR (400)
No hay suficientes habitaciones
disponibles. Disponibles: 2
(mensaje rojo)
- El formulario sigue relleno
- Puedes modificar los datos e intentar de
nuevo
phpMyAdmin (8081)
Puedes acceder a http://localhost:8081
para ver la BD directamente:
- Tabla hoteles: ver/modificar hoteles
- Tabla reservas: ver reservas creadas
- Tabla auditoria_reservas: ver historico